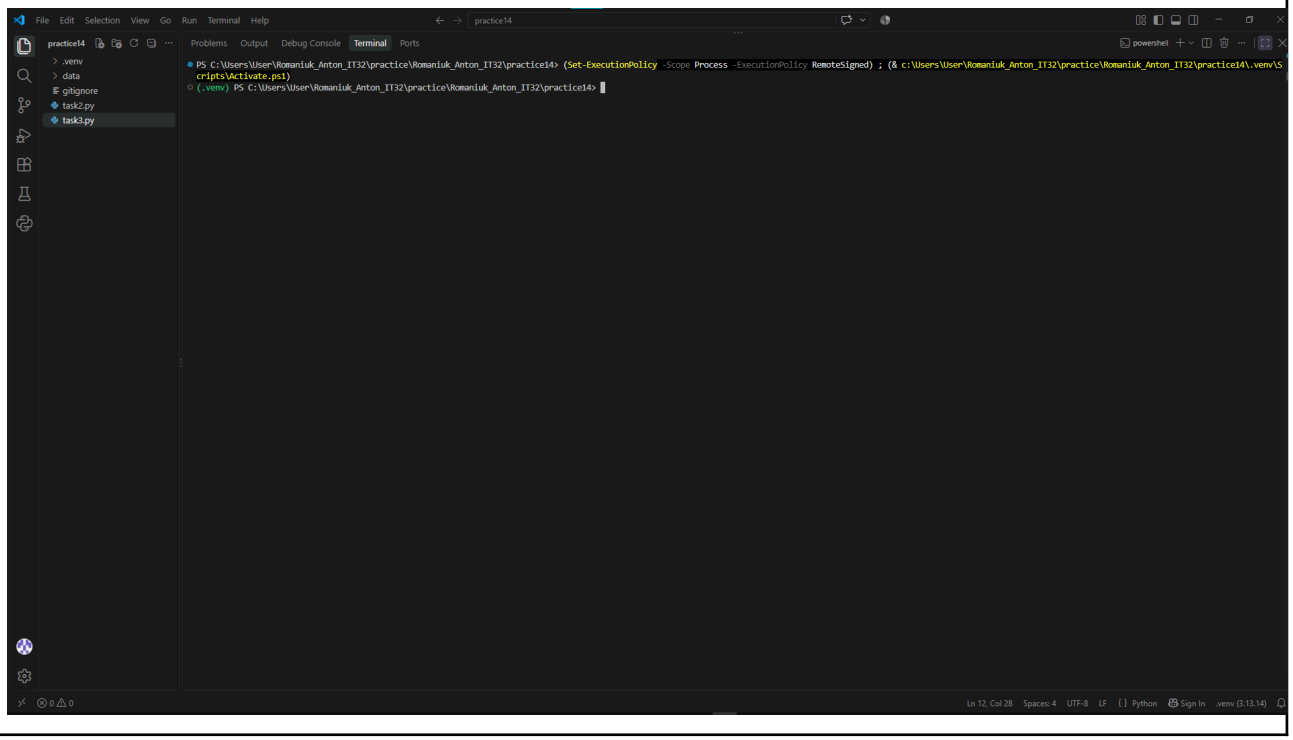
ЗВІТ з практичної роботи № 14

Студент: Anton Romaniuk

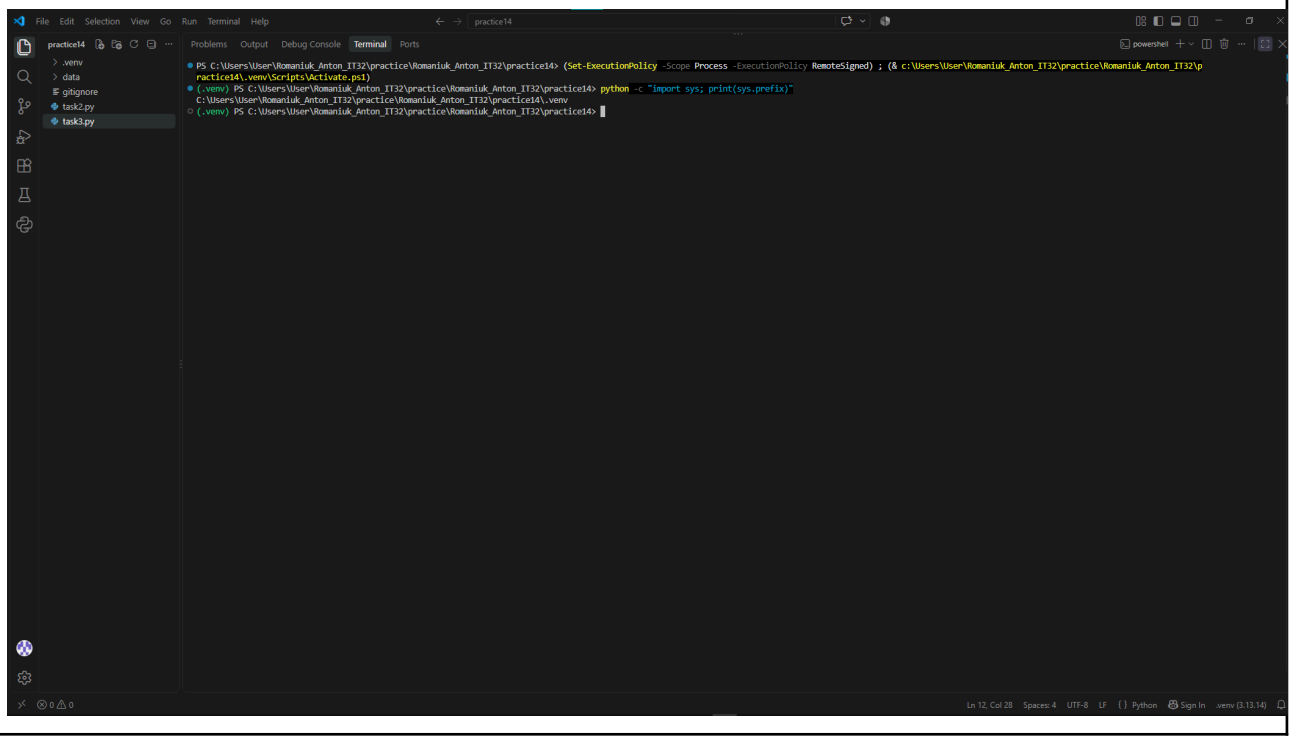
Група: IT-32

Завдання 1. Віртуальне середовище

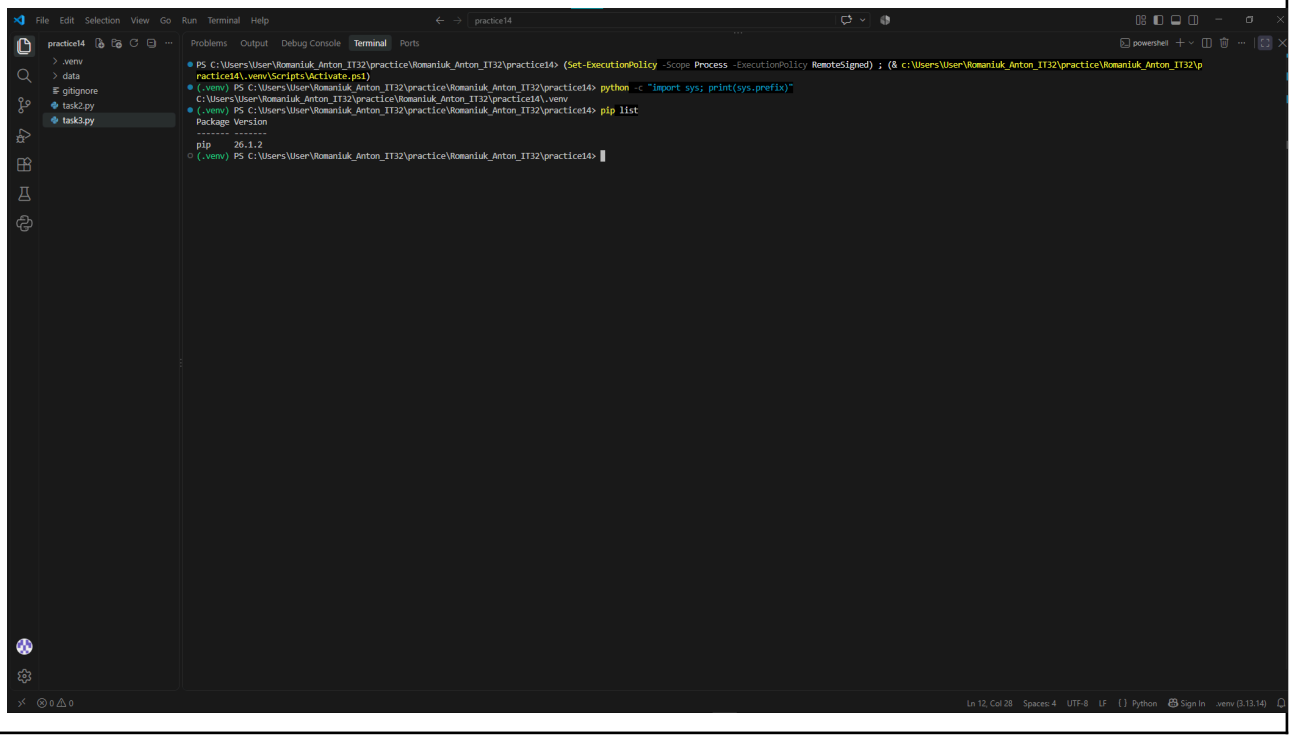
```
python3 -m venv .venv
.venv\Scripts\Activate.ps1
```



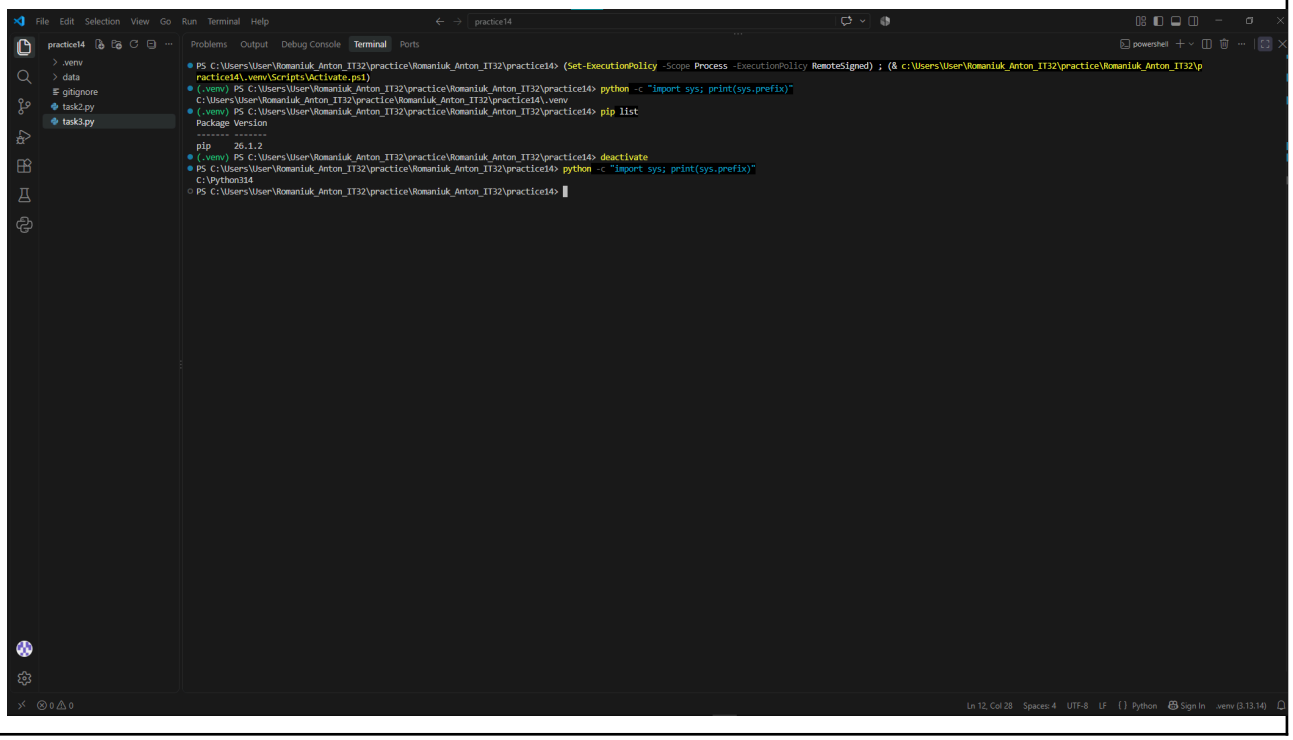
which python
python -c "import sys; print(sys.prefix)"



pip list



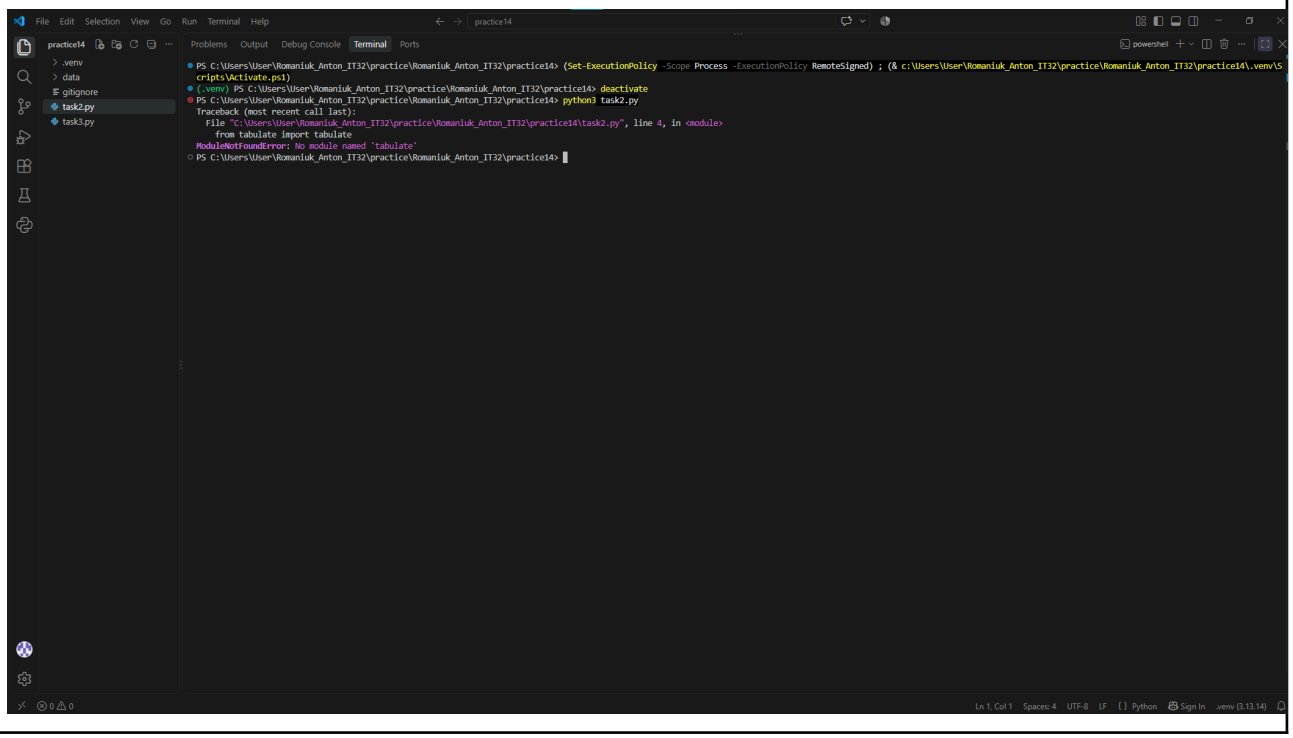
deactivate
which python



Завдання 2. Звіт про витрати з tabulate

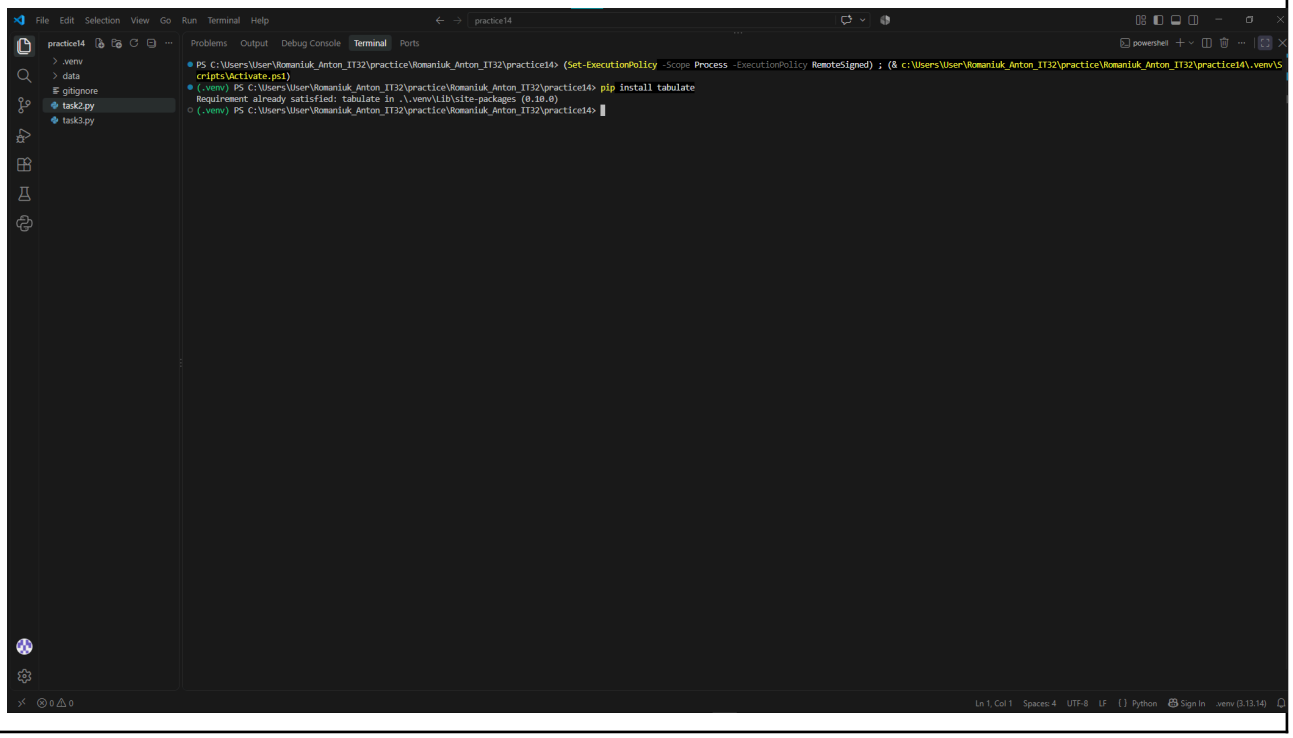
Програма task2.py читає data/expenses.csv (коментарі та порожні рядки пропускаються), групує суми за категоріями й виводить таблицю у форматі github з рядком TOTAL.

python task2.py # БЕЗ активного .venv

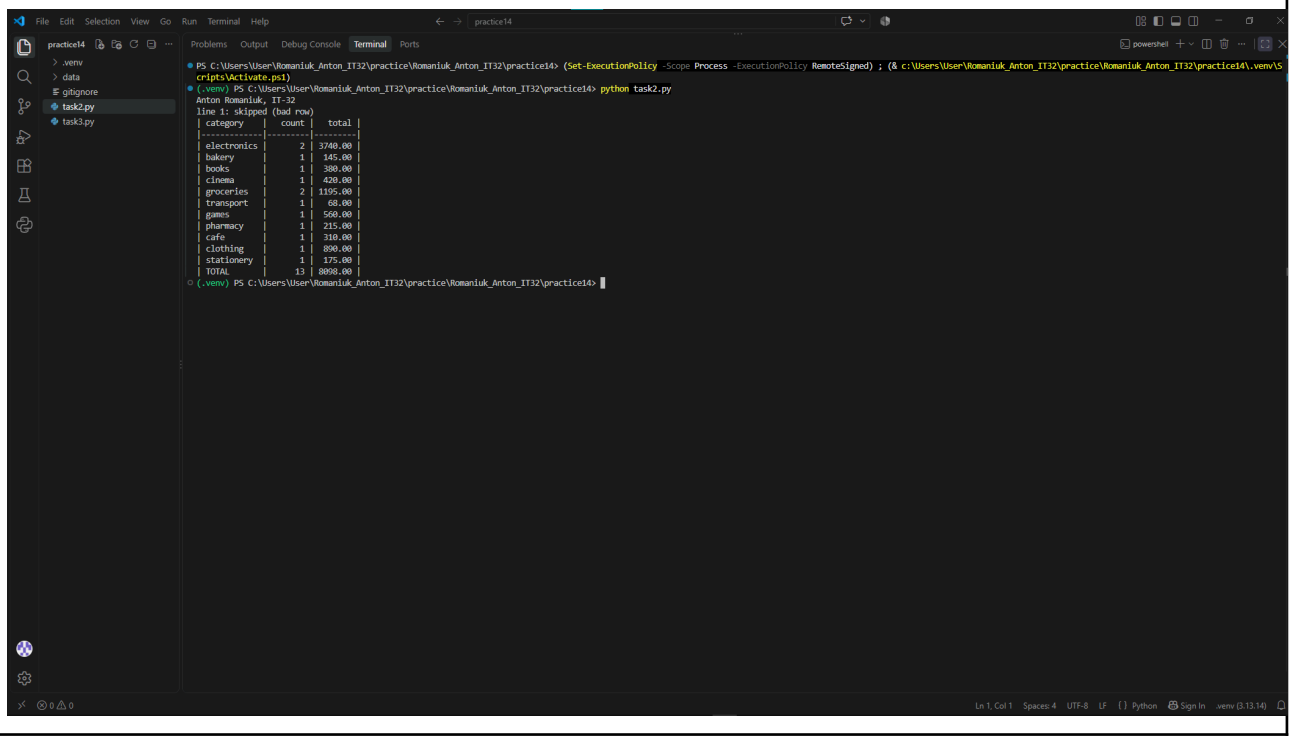


.venv\Scripts\Activate.ps1

pip install tabulate



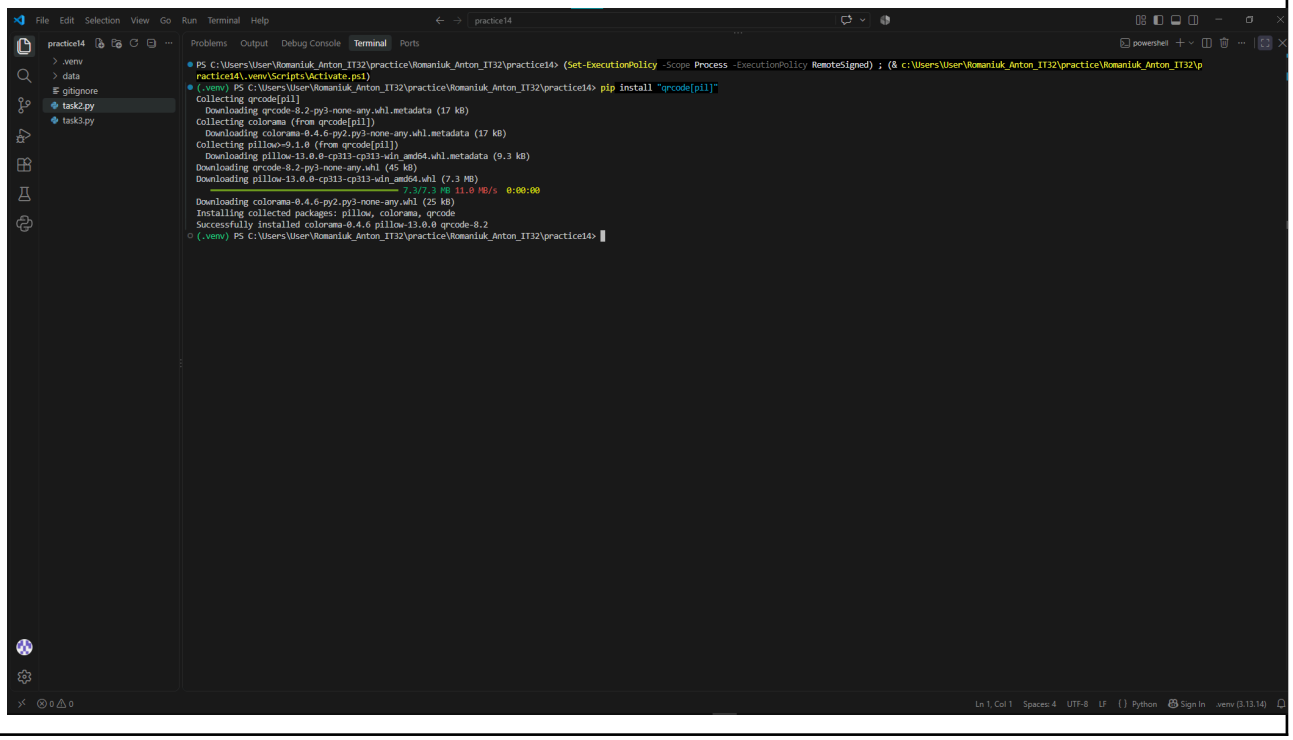
python task2.py # 3 активним .venv



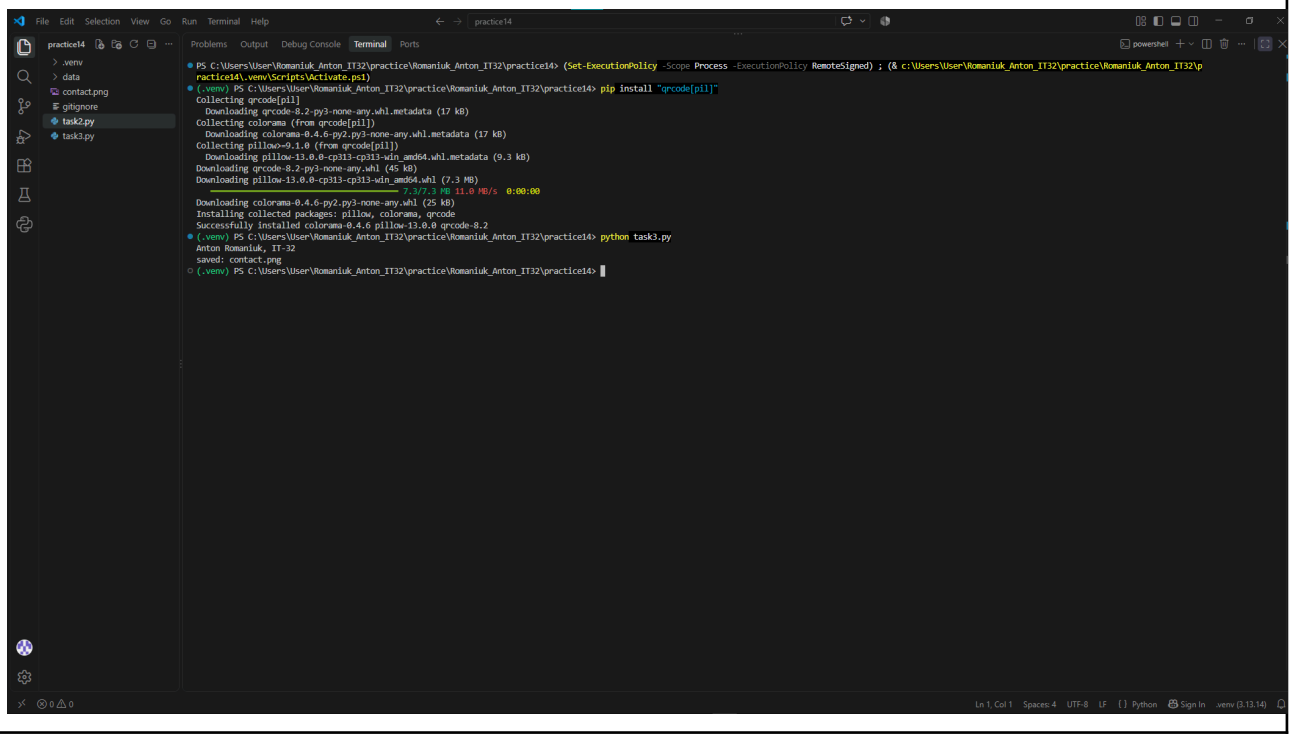
Завдання 3. QR-код з контактами

Програма task3.py створює файл contact.png з QR-кодом, що містить рядок «Anton Romaniuk, IT-32, <email>».

pip install "qrcode[pil]"



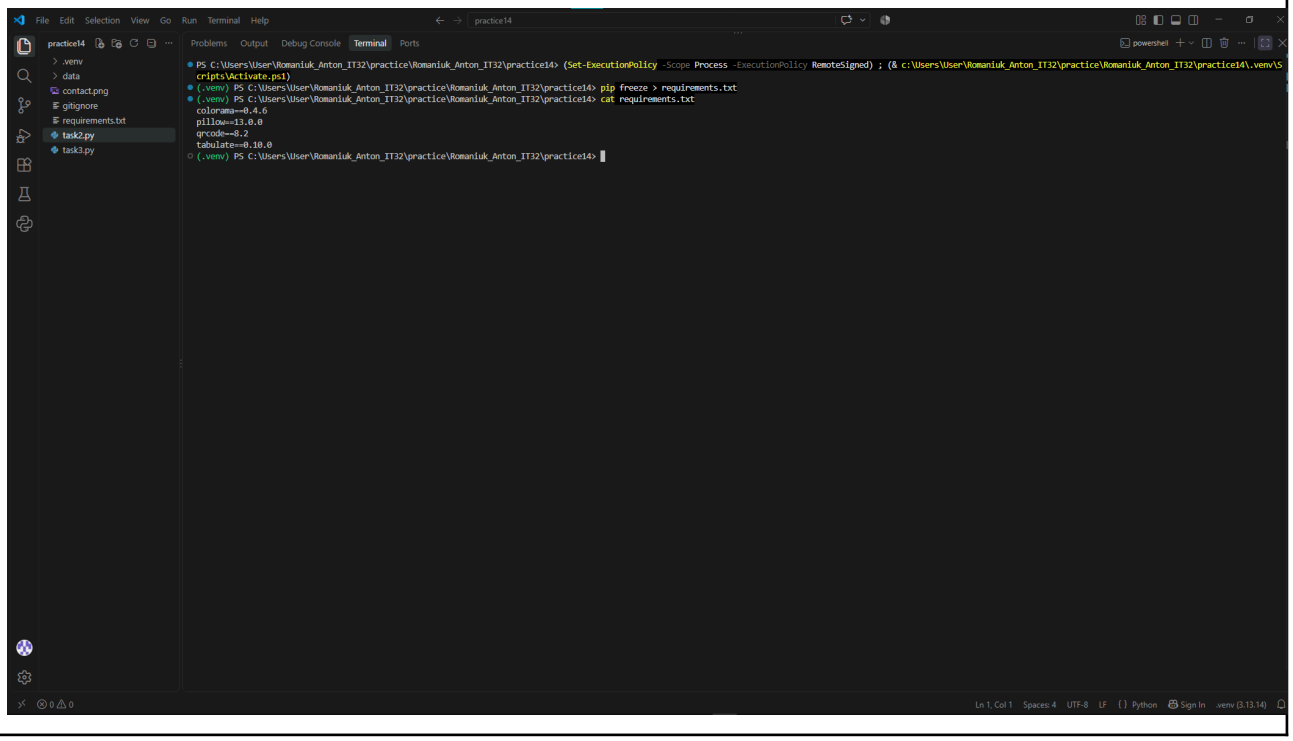
python task3.py



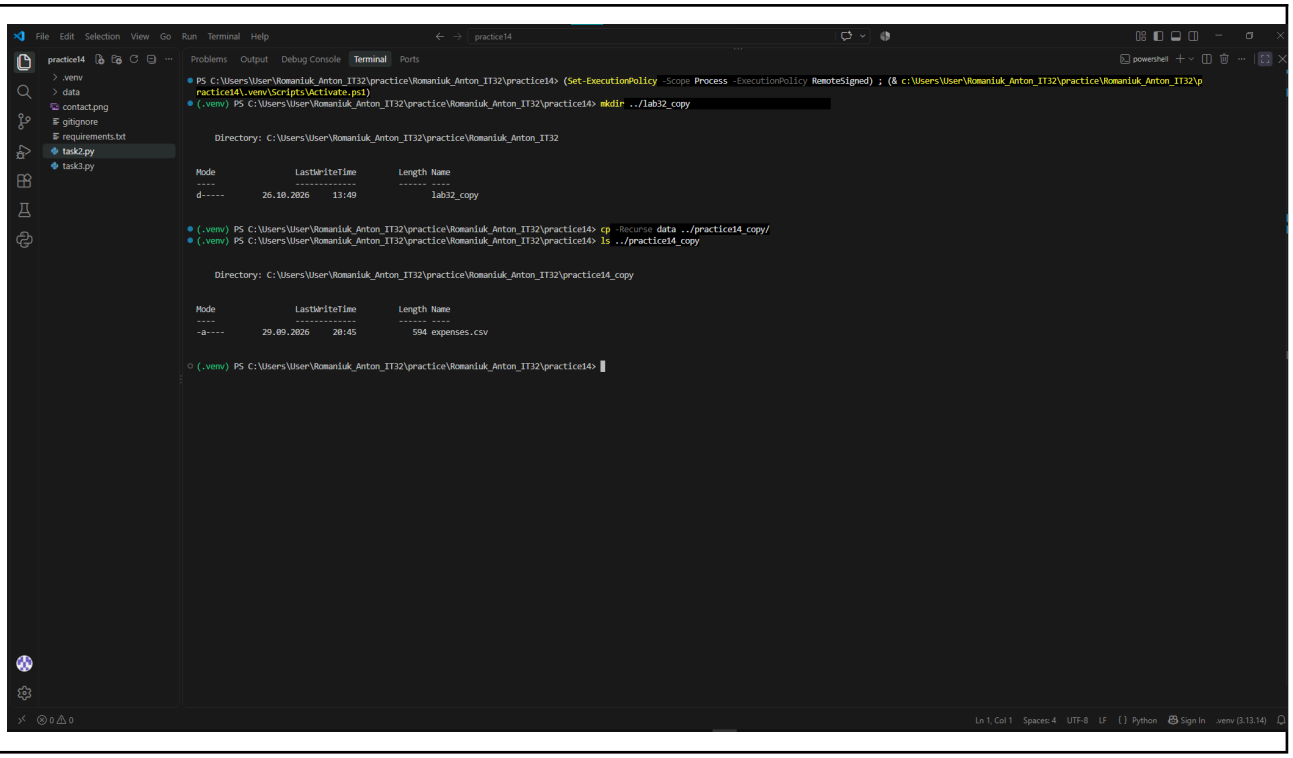


Завдання 4. Фіксація та відтворення залежностей

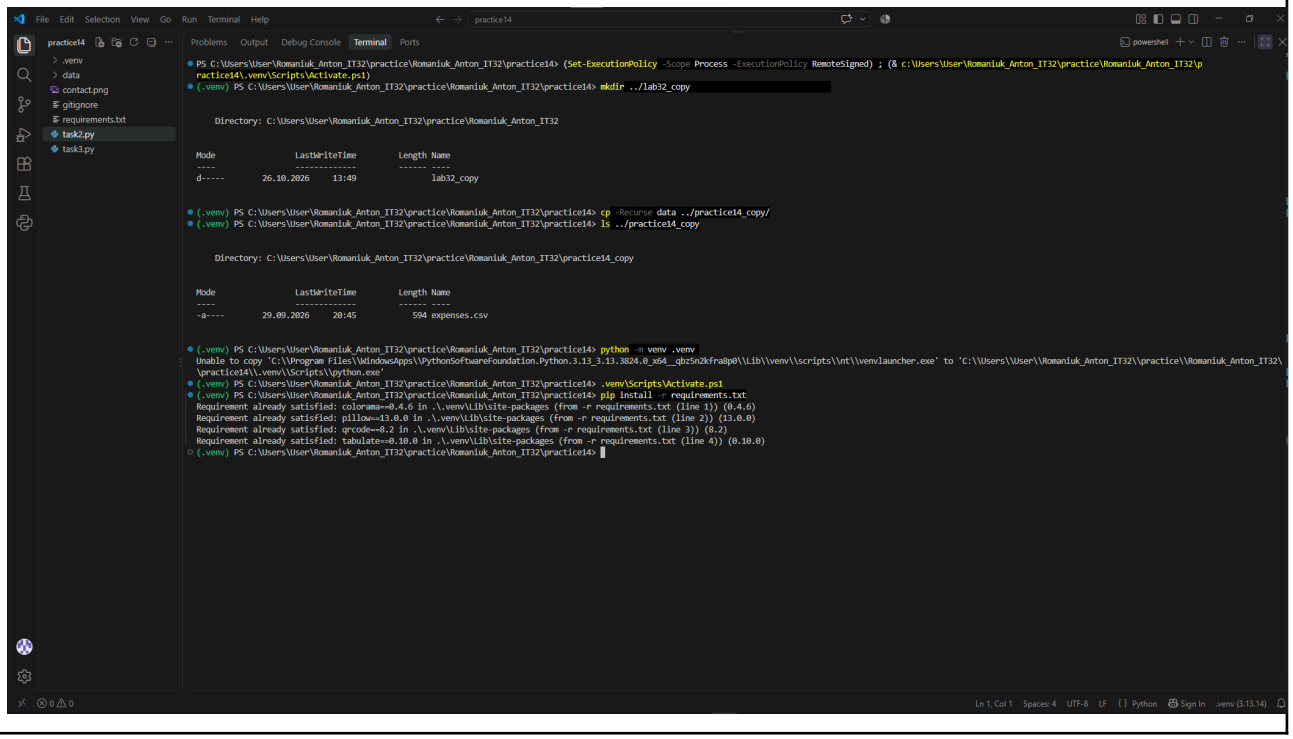
pip freeze > requirements.txt
cat requirements.txt



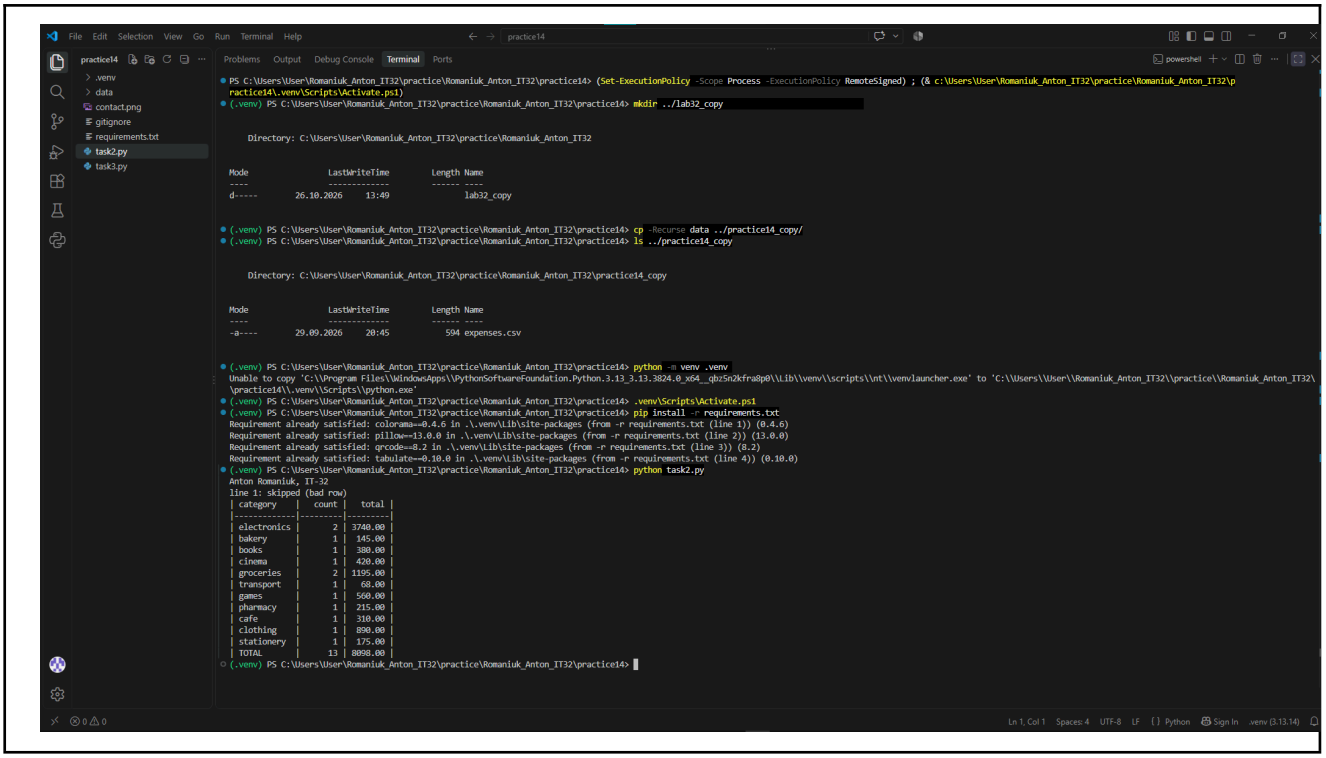
mkdir ../practice14_copy
cp task2.py, requirements.txt ../practice14_copy/
cp -Recurse data ../practice14_copy/
ls ../practice14_copy



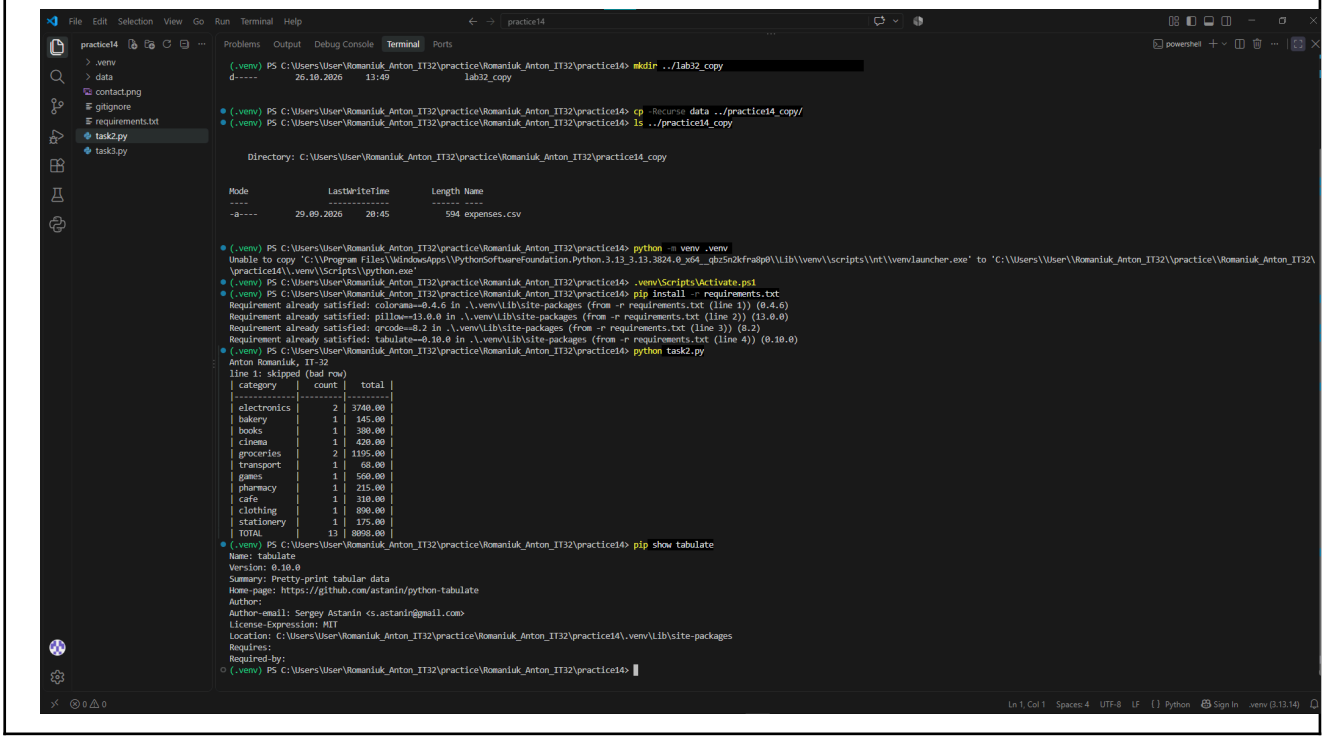
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt



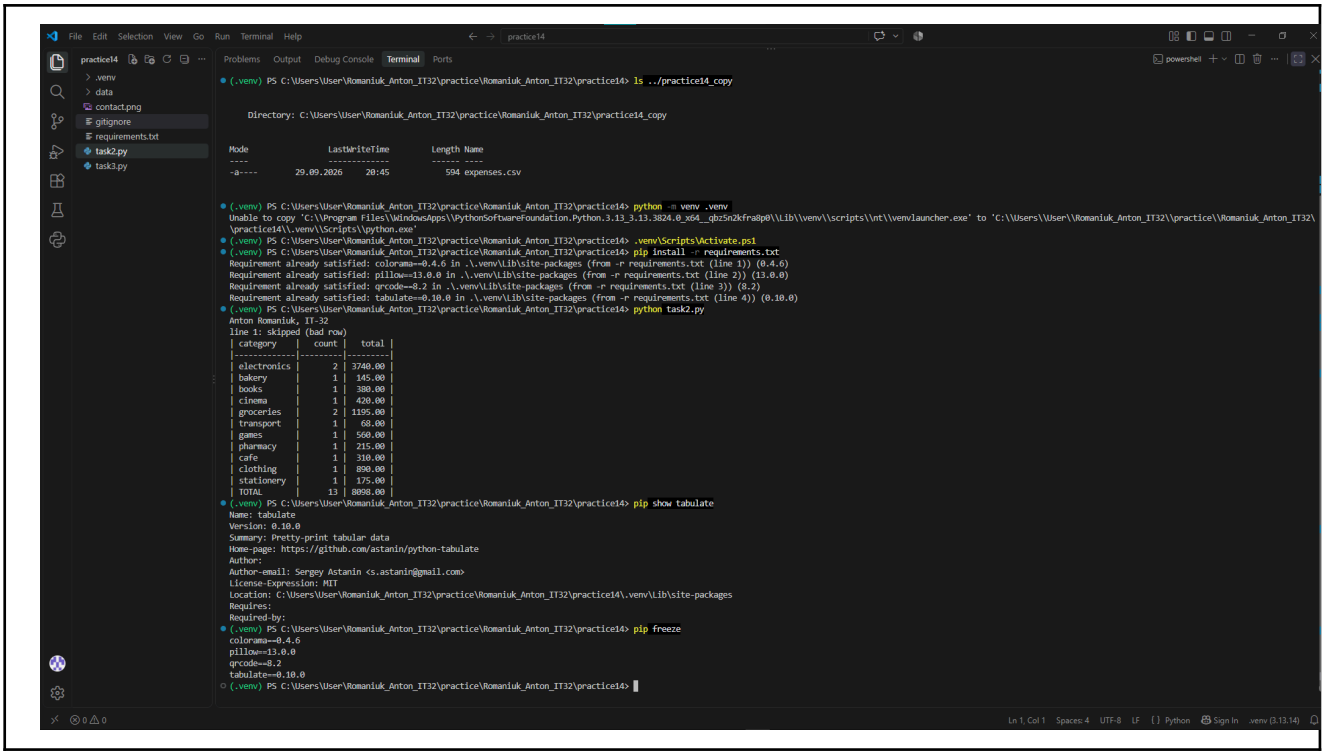
python task2.py



pip show tabulate



pip freeze

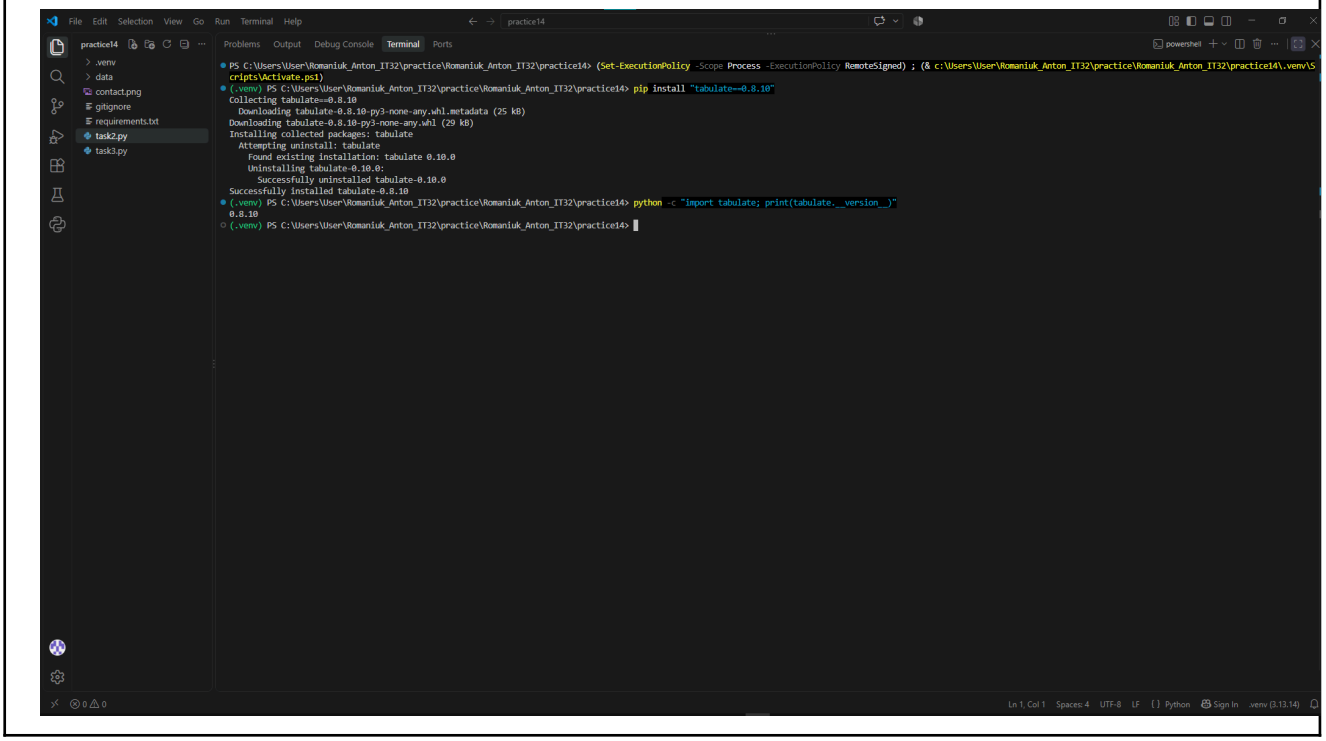


Чому в requirements.txt більше пакетів, ніж встановлено вручну? Команда pip freeze записує **всі** пакети середовища, зокрема транзитивні залежності: пакети, які автоматично встановилися разом із потрібними нам. Наприклад, qrcode[pil] тягне за собою pillow та інші. Так відтворене середовище матиме точно ті самі версії всього, що й оригінал.

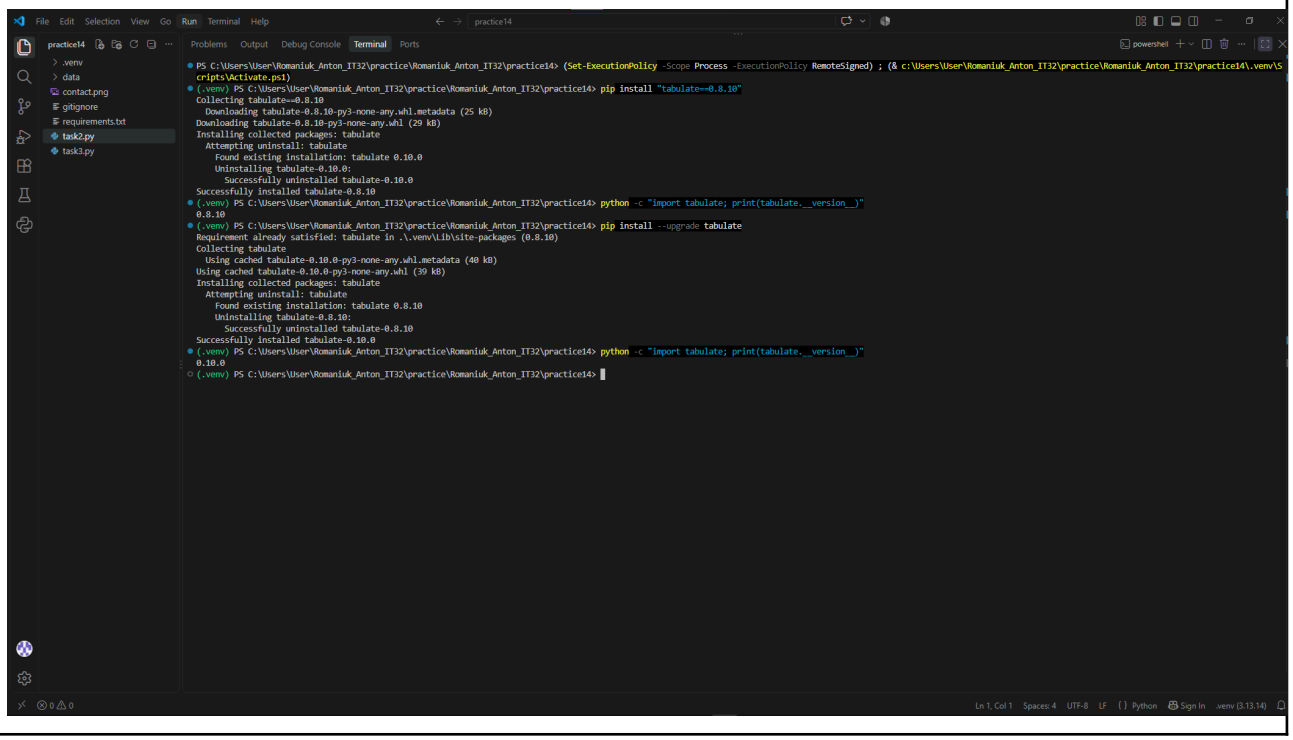
Завдання 5. Оновлення та видалення пакета

Команди виконуються в каталозі practice14 з активним .venv.

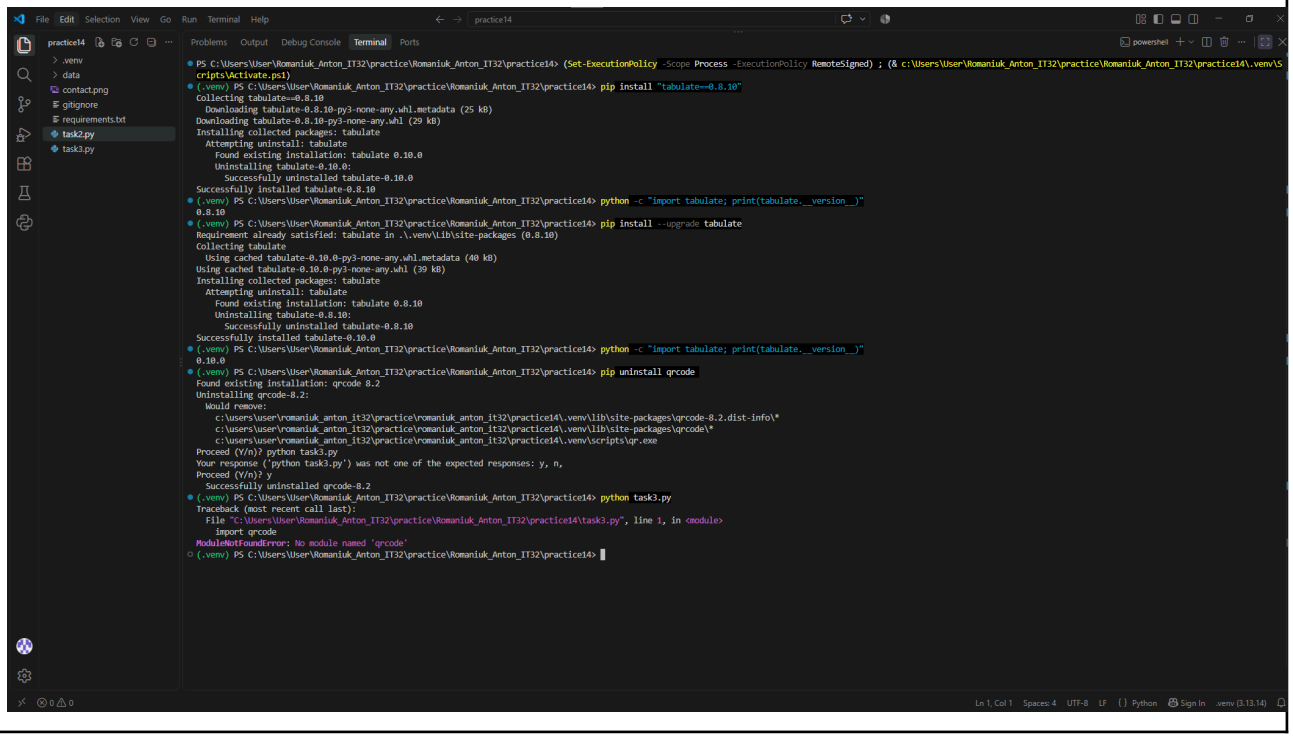
```
pip install "tabulate==0.8.10"
python -c "import tabulate; print(tabulate.__version__)"
```



pip install --upgrade tabulate
python -c "import tabulate; print(tabulate.__version__)"



pip uninstall qrcode
python task3.py



Відповіді на контрольні запитання

1. Навіщо потрібне віртуальне середовище і що станеться, якщо встановлювати всі пакети в системний Python?

Віртуальне середовище — це ізольований каталог із власним інтерпретатором і власним набором пакетів для одного проєкту. Воно потрібне, щоб:

• різні проєкти могли використовувати різні версії одного пакета без конфліктів; • не змінювати системний Python, від якого залежать інструменти операційної системи; • легко відтворити середовище (requirements.txt) і так само легко видалити його, просто стерши каталог.

Якщо ставити все в системний Python, версії пакетів різних проєктів конфліктують, оновлення одного проєкту може зламати інший або системні утиліти, у системі накопичується сміття, яке важко прибрати, а перелік залежностей конкретного проєкту неможливо відокремити від решти. У сучасних дистрибутивах Linux pip узагалі може відмовитися ставити пакети в системний Python (помилка externally-managed-environment).

2. Як переконатися, що активоване саме потрібне середовище? • у запрошенні терміналу є префікс (.venv);

• which python показує шлях усередині lab32/.venv/bin/;

• python -c "import sys; print(sys.prefix)" повертає шлях до lab32/.venv; • pip -V показує розташування pip всередині .venv.

3. Чим pip freeze відрізняється від pip list?

pip list виводить зручну для читання таблицю «пакет — версія». pip freeze друкує ті самі пакети у форматі ім'я==версія, придатному для requirements.txt, тому його перенаправляють у файл (pip freeze > requirements.txt) і потім відтворюють середовище через pip install -r. Також pip freeze за замовчуванням не показує службові пакети (pip, setuptools), а pip list показує.

4. Чому каталог .venv не додають до репозиторію, а requirements.txt — додають? • .venv великий і прив'язаний до конкретної машини: абсолютні шляхи, ОС, версія Python — на іншому комп'ютері він не запрацює;

• це згенерований вміст, який можна будь-коли відтворити, а чужий код у репозиторії лише його роздуває;

• requirements.txt — маленький текстовий файл, який описує потрібні пакети та їхні версії; за ним кожен створить власне ідентичне середовище. Тому .venv/ заносять до .gitignore, а requirements.txt комітять.

5. Що означає запис tabulate==0.9.0 і чим він відрізняється від tabulate>=0.9? tabulate==0.9.0 — потрібна **точно** версія 0.9.0, інші встановлені не будуть. Це дає відтворюваність: однакова поведінка в усіх середовищах. tabulate>=0.9 — підійде **будь-яка версія не нижча за 0.9**, зокрема нова мажорна, яка може змінити поведінку чи зламати програму. Це гнучкіше (отримуємо виправлення), але менш передбачувано. Для фіксації залежностей використовують ==.

6. Які ризики має встановлення пакета з неперевіреного джерела і як їх зменшити?

Пакет виконує код на вашому комп'ютері (під час встановлення та імпорту), тож ризики такі:

• шкідливий код (викрадення паролів, ключів, даних, бекдори);

• **typosquatting** — пакет із назвою, схожою на відому (наприклад, з помилкою в літері); • атаки на ланцюг постачання: скомпрометований автор, підміна внутрішніх залежностей, шкідливе оновлення;

• застарілі пакети з відомими вразливостями чи покинуті проєкти.

Як зменшити ризики:

• встановлювати лише з офіційного PyPI й перевіряти точну назву пакета, автора, репозиторій, історію релізів і популярність;

• ставити пакети тільки у віртуальне середовище, ніколи через sudo pip у системний Python;

• фіксувати версії (==), за потреби — перевіряти хеші (--require-hashes); • мінімізувати кількість залежностей і регулярно перевіряти їх на вразливості (наприклад, pip-audit);

• не запускати код з випадкових джерел і не ставити пакети за порадами з невідомих сайтів чи чатів.

Безпека програмного забезпечення — частина кібербезпеки держави, тому відповідальне ставлення до сторонніх залежностей є обов'язком розробника.